Feature Vector Class Specs

A FeatureVector is an object that allows for set schemas with different attributes. Each attribute will be represented by a floating-point number.

Goal:

- An index can be accessed by name quickly
- Support dot product, weighted scoring, normalization, and other common operations
- No adding features at runtime in each vector.

Functionality:

- Construct with a schema, and have initial values set to 0
- Read and write via index or name lookup
- Compute operations listed above efficiently

Similar standard library classes:

- Std::unordered_map<std::string, size_t>, similar, but doesn't allow for schema mapping to be shared throughout multiple vectors/objects
- Std::vector<double>. Doesn't allow name lookup or schema sharing, but does allow individual doubles to be stored at index. Additionally, is malleable at runtime

Key functions:

explicit FeatureVector(std::shared_ptr<const FeatureSchema> schema);

- Stored as a shared_ptr to avoid dangling
- Initially all 0

const FeatureSchema& Schema() const
void Clear() (just set all values to 0)
size_t Size() const
double Get(size_t i) const
void Set(size_t i, double v)
bool TryGet(std::string_view name, double& out) const
bool TrySet(std::string_view name, double v)
Additions may be made for operations as they seem helpful

Error Conditions:

- Index out of range
- Attempting to perform operations on schemas that aren't the same

- Search name not found in schema (TryGet(name), TrySet(name)
    - Just return false, no need to throw anything

Expected challenges:

- Whether it should be stored as a vector of floats vs double (memory vs precision)
- Making sure its numerically stable

Other Groups' to coordinate with:

- Primarily other group 5 classes:
    - RobinHoodMap could be used for a schema name to index mapping
- Outside of group 5, potentially:
    - Group 23: DataLog class, keeping feature snapshots
    - Group 9: Serializer, if we want to standardize serialization and include that functionality in FeatureVector's implementation

Annotation Set Class Specs

An annotation set is a set of tags that are connected to a defined object. Each tag will be a string that contains information about the environment that the ai agent will learn from.

Goals:

- Stored in a way for quick tag lookup
- Support adding and removing tags quickly
- Support finding specific tags

Functionality

- Being able to have an object be connected to multiple tags
- Support deleting and adding tags to certain objects

Similar Standard Library Classes

- Std::pair<int, std::vector<std::string>>
    - Very similar but doesnt allow for looking up tags and also doesnt allow for looking up more than one tag

Key functions

- AnotationSet(int, std::vector<std::string>)
    - Int will be the object id and the vector will be initial tags
- AnnotationSet(int)
    - Constructor for a set with no tags (object must always be provided)
- Void AddTag(std::string)
    - Simple function to add a tag to the set
- bool RemoveTag(std::string)
    - Removes a tag from the set
- Bool FindTag(std::string)
    - Checks if the tag is in the set
- Bool FindAnyTag(std::vector<std::string>)
    - Checks if any tag in the vector is in the set
- Bool FindAllTags(std::vector<std::string>)
    - Checks if all tags in the vector are in the set
- Void

Error Conditions

- Adding an already existing tag
    - Could deal with this by using a set to keep tags thereby avoiding duplicates
- Removing a tag that doesnt exist
    - Dealing with this using a bool just in case

Other groups to coordinate with

- In group 5
    - tagManager since they will in charge of looking for objects with a certain tag through annotation sets

RobinHoodMap Specs

Class description:

high-performance associative container. Faster alternative to std::unordered_map for workloads with many inserts/lookups. when collisions occur, entries steal spots from entries that are closer to their home bucket, reducing variance in probe length and improving lookup speed.

Key goals

- Emscripten-friendly C++
- Fast find/contains and insert/emplace (cache-friendly)
- Predictable performance: load factor control and robin-hood probe strategy.

Similar standard library classes

- std::unordered_map (hash map API expectations, load factor, rehashing)
- std::vector (contiguous bucket storage)
- std::hash, std::equal_to (hashing + key equality defaults)
- std::pair, std::tuple (return types for insert operations)

Key functions

- template<class Key, class T,
  class Hash = std::hash<Key>,
  class KeyEq = std::equal_to<Key>>
- class RobinHoodMap;

Construction / capacity

- RobinHoodMap()
- explicit RobinHoodMap(size_t initial_capacity)
- size_t Size() const
- bool Empty() const
- void Clear()
- void Reserve(size_t n)
  - Ensure space for ~n items without frequent rehashing
- void Rehash(size_t bucket_count)
- float LoadFactor() const
- void MaxLoadFactor(float f) / float MaxLoadFactor() const
- size_t BucketCount() const (optional)

Lookup

- T* Find(const Key& key) / const T* Find(const Key& key) const
- bool Contains(const Key& key) const
- T& At(const Key& key) / const T& At(const Key& key) const
- T& operator[](const Key& key) (inserts default value if missing)

Insert / update

- std::pair<T*, bool> Insert(const Key& key, const T& value)
- std::pair<T*, bool> InsertOrAssign(const Key& key, T value)
- template<class... Args> std::pair<T*, bool> Emplace(Key key, Args&&... args)

Erase

- bool Erase(const Key& key) (returns false if not found)

Error Conditions:

- Index / bucket misuse (internal invariants)
- Programmer error → assert (should never occur in correct usage)
- At(key) on missing key

- Recoverable error → throw std::out_of_range (matches STL style)
- Invalid load factor (e.g., MaxLoadFactor(f <= 0 or >= 1))
- Programmer error → assert (or clamp, but assert is clearer for specs)
- Allocation failure during Reserve/Rehash
- Recoverable error → std::bad_alloc (standard behavior)
- Key not found for Find/Contains/Erase
- Normal condition → Find returns nullptr, Contains false, Erase false

Expected challenges:

- Correct Robin Hood insertion logic (swap based on probe distance)
- Deletion strategy:
- backward-shift deletion (preferred for performance)
- Managing rehash thresholds and sizing strategy
- Handling move-only keys/values and perfect forwarding in Emplace
- Clearly documenting and testing invalidation behavior

other group's C++ classes

- Primarily other Group 5 classes:
    - TagManager / AnnotationSet will likely use RobinHoodMap internally for fast tag → entity IDs and entity ID → annotations lookups
- • Outside of Group 5, potentially:
    - Group 6: ActionMap / BehaviorTree "blackboard" could use RobinHoodMap for fast string → function/value mapping
    - Group 7/8: World modules may use RobinHoodMap for entity registries (e.g., EntityId → state) and quick indexing
    - Group 9: Database module may want RobinHoodMap for in-memory indices/caches (IDs → records) before serialization

Function Set Specs

Function Set will contain a collection of callable objects that will share the same signature. The caller can invoke the entire set with one call, and Funtion Set will execute each stored function one-by-one with the provided arguments.

Key Goals
Support extensibility: add or remove behaviors without changing calling code
Provide predictable iteration order and simple error handling
Emscripten-friendly C++
Simple, low-overhead storage and invocation
Works with lambdas that capture state

Similar standard library classes / concepts
std::function for type-erased callable storage
std::vector for contiguous storage of callbacks

Key functions and API shape
FunctionSet is templated on a function signature, for example:
FunctionSet<void(int, double)>

FunctionSet()
size_t Size() const
bool Empty() const
void Clear()

void Reserve(size_t n) (optional optimization)

Add(function) : returns a handle identifying the stored function
Supports std::function or generic callables via perfect forwarding

Remove(handle) → removes a previously added function
Returns false if the handle is invalid or already removed

CallAll(args...)
For void-returning functions: executes all functions sequentially
For non-void functions: returns a vector of results in execution order

Return behavior

If the function signature returns void, CallAll returns nothing
If the function signature returns a value R, CallAll returns a vector<R>
Results are returned in the order the functions were added

Error conditions and behavior
Calling an empty FunctionSet performs no work
Removing a non-existent function returns false

Programmer error (assert)

Invalid internal state (e.g., corrupted handle)
Index out of bounds if index-based removal is used

Recoverable runtime errors

Exceptions thrown by stored functions propagate by default

Expected challenges
Designing a safe removal mechanism when multiple systems register callbacks
Deciding between stable iteration order and O(1) removal
Handling callable copyability (std::function typically requires copyable callables)
Clearly documenting whether execution order is guaranteed
Relationship to other Group 5 classes and Outside Group 5

FunctionSet can hold evaluators that produce FeatureVectors
FunctionSet can hold scorers that combine FeatureVectors with agent tendencies
FunctionSet can hold post-action update logic for experience-driven learning

Group 6 (Classic Agents): behavior-tree hooks or blackboard update callbacks
Group 7/8 (World modules): event listeners for interactions or construction events
Group 9 (Database): triggers or observers for in-memory state updates
Group 24 (Web Interface): UI or simulation event callback aggregation

TagManager (Group 5: AI Agents)

TagManager

class description: tag manager keep track of which objects have what tags. It works with AnnotationSet. Each object stores its own tags, and TagManager helps quickly find all objects that share certain tags.

Goals:

- Find objects by tag quickly
- Support searching with multiple tags
- Avoid pointer or lifetime issues by using object IDs

Functionality:

- Add tag to an object
- Remove tag from an object
- Find objects with:
- One tag
- Multiple required tags
- Required tags but excluding others
- Remove a object from all tags when it's deleted

Similar standard library classes:

- std::unordered_map<std::string, std::vector<ObjectID>>
- std::unordered_set<std::string>
- std::vector<ObjectID>
- RobinHoodMap

Key functions:

- TagManager();
- void Clear();
- void RemoveObject(ObjectId id);
- bool AddTag(ObjectId id, std::string_view tag);
- bool RemoveTag(ObjectId id, std::string_view tag);
- bool HasTag(ObjectId id, std::string_view tag) const;
- size_t TagSize(std::string_view tag) const;
- std::vector<ObjectId> GetWithTag(std::string_view tag) const;
- std::vector<ObjectId> QueryAll(
  std::span<const std::string_view>) const;
- std::vector<ObjectId> QueryAllNot(
  std::span<const std::string_view>,
  std::span<const std::string_view>) const;};

Error Conditions:

- Tag string empty
- Programmer error -> assert
- Object ID invalid
- Programmer error -> assert
- Removing an unpresent tag

- Normal case -> return false
- Searching for an unexisted tag
- Normal case -> return empty list
- Memory allocation fails
- Recoverable error -> std::bad_alloc

Expected challenges:

- Making tag search fast when a lot of objects exist
- Efficiently removing object IDs from tag list
- Keeping object -> tag and tag -> object data in sync
- Making sure strings are safely stored
- Documenting clearly when results may change

Groups to coordinate with:

- Group 5
    - AnnotationSet
    - RobinHoodMap
- Other Groups:
    - Group 7/8 (World Modules) - deciding which objects to be tagged
    - Group 9 (Database) - saving and loading the tags
    - Group 24 (Web Interface) - Showing and filtering tags in UI
    - Group 23 (Data Analytics) - Analysing and logging usage of tag